

13.1 Introducción

La persistencia de datos constituye un aspecto fundamental en el desarrollo de aplicaciones. A lo largo de los capítulos precedentes se ha trabajado con archivos como mecanismo para conservar información; sin embargo, cuando se necesita almacenar grandes volúmenes de datos estructurados, efectuar consultas complejas o preservar la integridad de la información, el empleo de una base de datos relacional se presenta como la alternativa más adecuada.

Python ofrece interfaces para conectarse con la mayoría de los sistemas de gestión de bases de datos relacionales. Algunas de estas interfaces forman parte de la biblioteca estándar, mientras que para los motores restantes existen bibliotecas que pueden instalarse mediante gestores de paquetes como `pip`. Python incorpora en su biblioteca estándar el módulo `sqlite3`, que ofrece una interfaz para trabajar con bases de datos SQLite.

13.2 SQLite

SQLite es un sistema de gestión de bases de datos relacional que no requiere de un servidor independiente, ya que la base de datos completa se almacena en un único archivo. Esta característica la convierte en una alternativa especialmente conveniente para aplicaciones de escritorio, prototipos, aplicaciones móviles y, en general, para todo escenario en el que no se requiera concurrencia masiva.

Se trata de un sistema autocontenido, que no requiere instalación ni configuración de servidor y funciona directamente, sin archivos de configuración. Es transaccional, dado que cumple las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), y resulta portable, puesto que la base de datos es un archivo único que puede copiarse entre sistemas. Por último, implementa la mayor parte del estándar SQL, lo que la convierte en una opción completa para la mayoría de las aplicaciones.

13.3 Conexión a una base de datos

Para trabajar con una base de datos SQLite en Python, es necesario primero importar el módulo y establecer una conexión:

```
import sqlite3

# Conectar a una base de datos (se crea si no existe)
conexion = sqlite3.connect('mi_base_datos.db')

# Crear un cursor para ejecutar comandos SQL
cursor = conexion.cursor()

# Realizar operaciones...

# Cerrar la conexión
conexion.close()
```

El método `connect()` recibe como parámetro el nombre del archivo de base de datos. Si el archivo no existe, SQLite lo crea automáticamente. Para crear una base de datos en memoria (temporal), se utiliza el nombre especial `':memory:'`.

Buena práctica: uso de context managers Es recomendable utilizar la sentencia `with`, ya que garantiza que la conexión se cierre correctamente aun cuando ocurra un error durante la ejecución de las operaciones:

```
import sqlite3

with sqlite3.connect('mi_base_datos.db') as conexion:
    cursor = conexion.cursor()
    # Realizar operaciones...
# La conexión se cierra automáticamente
```

13.4 Objeto cursor

El cursor es el objeto a través del cual se ejecutan las sentencias SQL sobre la base de datos y se recuperan los resultados obtenidos. Se obtiene a partir de la conexión mediante el método `cursor()`. Sus principales métodos son:

- `execute(sql, parametros=None)`: Ejecuta una sentencia SQL; el segundo argumento, opcional, permite incorporar valores a la consulta por medio de parámetros
- `executemany(sql, secuencia)`: Ejecuta la misma sentencia una vez por cada conjunto de valores contenido en la secuencia
- `fetchone()`: Devuelve la siguiente fila del resultado como una tupla, o `None` si no hay más filas
- `fetchall()`: Devuelve todas las filas restantes como una lista de tuplas
- `fetchmany(tam)`: Devuelve hasta `tam` filas como una lista de tuplas

Además de estos métodos, el cursor posee dos atributos de particular utilidad. `lastrowid` almacena el identificador asignado al último registro insertado mediante ese cursor, en tanto que `rowcount` informa la cantidad de filas afectadas por la última operación ejecutada.

El siguiente ejemplo ilustra el empleo de los métodos de recuperación de filas:

```
import sqlite3

with sqlite3.connect('personas.db') as conexion:
    cursor = conexion.cursor()

    cursor.execute('SELECT * FROM personas')

    # Obtener fila por fila
    print("Primera persona:")
    fila = cursor.fetchone()
    print(fila)

    print("\nSiguientes 2 personas:")
    filas = cursor.fetchmany(2)
    for fila in filas:
        print(fila)

    print("\nRestantes:")
    filas = cursor.fetchall()
    for fila in filas:
        print(fila)
```

13.5 Consultas

Las consultas son las sentencias SQL orientadas a la recuperación de datos. Se estructuran a partir de SELECT y devuelven las filas que satisfacen los criterios indicados. Para recuperar todos los registros de una tabla:

```
import sqlite3

with sqlite3.connect('personas.db') as conexion:
    cursor = conexion.cursor()

    # Consultar todos los registros
    cursor.execute('SELECT * FROM personas')

    # Obtener todos los resultados
    resultados = cursor.fetchall()

    for fila in resultados:
        print(fila)
```

13.5.1 Consultas con filtros

La cláusula WHERE permite limitar los resultados de una consulta a las filas que cumplen una condición determinada, mientras que la cláusula ORDER BY ordena los resultados según una columna:

```
import sqlite3

def buscar_por_documento(documento):
    with sqlite3.connect('personas.db') as conexion:
        cursor = conexion.cursor()

        cursor.execute('''
            SELECT * FROM personas WHERE documento = ?
        ''', (documento,))

        return cursor.fetchone()

def buscar_mayores_de(edad_minima):
    with sqlite3.connect('personas.db') as conexion:
        cursor = conexion.cursor()

        cursor.execute('''
            SELECT * FROM personas WHERE edad ≥ ?
            ORDER BY edad DESC
        ''', (edad_minima,))

        return cursor.fetchall()

# Uso
persona = buscar_por_documento('12345678')
print(f"Persona encontrada: {persona}")
```

```
mayores = buscar_mayores_de(30)
print(f"\nPersonas mayores de 30 años:")
for p in mayores:
    print(p)
```

13.5.2 Trabajar con diccionarios en lugar de tuplas

Por defecto, el módulo `sqlite3` devuelve las filas como tuplas. Para acceder a los datos por el nombre de la columna, es posible configurar la propiedad `row_factory` de la conexión asignándole la clase `sqlite3.Row`: de este modo, cada fila se comporta como una estructura similar a un diccionario, y los valores pueden consultarse mediante el nombre de la columna:

```
import sqlite3

with sqlite3.connect('personas.db') as conexion:
    # Configurar para devolver diccionarios
    conexion.row_factory = sqlite3.Row
    cursor = conexion.cursor()

    cursor.execute('SELECT * FROM personas')

    for fila in cursor.fetchall():
        # Acceder por nombre de columna
        print(f"{fila['nombre']} {fila['apellido']}, "
              f"{fila['edad']} años, DNI: {fila['documento']}")
```

13.6 Sentencias de modificación de datos

Las sentencias `INSERT`, `UPDATE` y `DELETE` permiten crear, modificar y eliminar los datos almacenados en las tablas. A diferencia de las consultas, estas sentencias no devuelven filas como resultado. Asimismo los cambios que producen deben confirmarse con `commit()` para volverse permanentes.

13.6.1 Inserción de datos

Para insertar registros se utiliza la sentencia `INSERT INTO`:

```
import sqlite3

with sqlite3.connect('personas.db') as conexion:
    cursor = conexion.cursor()

    # Insertar un registro
    cursor.execute('''
        INSERT INTO personas (documento, nombre, apellido, edad)
```

```
        VALUES ('12345678', 'Juan', 'Pérez', 30)
    ''')

    conexion.commit()
    print(f"Registro insertado con ID: {cursor.lastrowid}")
```

El atributo `lastrowid` del cursor almacena el identificador correspondiente al último registro insertado.

13.6.2 Inserción con parámetros

Nunca deben construirse consultas SQL concatenando cadenas con valores proporcionados por el usuario, ya que esta práctica expone a la aplicación a ataques de *inyección SQL*. En lugar de ello, los valores deben incorporarse a la consulta mediante parámetros:

```
import sqlite3

def insertar_persona(documento, nombre, apellido, edad):
    with sqlite3.connect('personas.db') as conexion:
        cursor = conexion.cursor()

        # Uso de parámetros (placeholders con ?)
        cursor.execute('''
            INSERT INTO personas (documento, nombre, apellido, edad)
            VALUES (?, ?, ?, ?)
        ''', (documento, nombre, apellido, edad))

        conexion.commit()
        return cursor.lastrowid

# Uso de la función
id_persona = insertar_persona('87654321', 'María', 'González', 25)
print(f"Persona insertada con ID: {id_persona}")
```

Los signos de interrogación `?` funcionan como marcadores de posición (*placeholders*) que son reemplazados por los valores de la tupla enviada como segundo argumento.

13.6.3 Inserción múltiple

Para insertar varios registros en una misma operación, se utiliza el método `executemany()`, que recibe la consulta y una secuencia de tuplas con los valores a insertar:

```python
import sqlite3

personas = [
    ('11111111', 'Pedro', 'Martínez', 35),
    ('22222222', 'Ana', 'López', 28),
    ('33333333', 'Luis', 'Rodríguez', 42)
]

with sqlite3.connect('personas.db') as conexion:
    cursor = conexion.cursor()

    cursor.executemany('''
        INSERT INTO personas (documento, nombre, apellido, edad)
        VALUES (?, ?, ?, ?)
    ''', personas)

    conexion.commit()
    print(f"{cursor.rowcount} registros insertados")
```

13.6.4 Actualización de datos

Para modificar registros existentes se utiliza la sentencia `UPDATE`:

```python
import sqlite3

def actualizar_edad(documento, nueva_edad):
    with sqlite3.connect('personas.db') as conexion:
        cursor = conexion.cursor()

        cursor.execute('''
            UPDATE personas
            SET edad = ?
            WHERE documento = ?
        ''', (nueva_edad, documento))

        conexion.commit()

        if cursor.rowcount > 0:
            print(f"Edad actualizada para documento {documento}")
        else:
            print(f"No se encontró persona con documento {documento}")

# Uso
actualizar_edad('12345678', 31)
```

El atributo `rowcount` del cursor indica la cantidad de filas afectadas por la última operación ejecutada. Su consulta resulta útil para determinar, por ejemplo, si una actualización alcanzó efectivamente algún registro.

13.6.5 Eliminación de datos

Para eliminar registros se utiliza la sentencia `DELETE`:

```
import sqlite3

def eliminar_persona(documento):
    with sqlite3.connect('personas.db') as conexion:
        cursor = conexion.cursor()

        cursor.execute('''
            DELETE FROM personas
            WHERE documento = ?
        ''', (documento,))

        conexion.commit()

        if cursor.rowcount > 0:
            print(f"Persona con documento {documento} eliminada")
        else:
            print(f"No se encontró persona con documento {documento}")

# Uso
eliminar_persona('12345678')
```

13.7 Transacciones

Una transacción es un conjunto de operaciones que se ejecutan como una única unidad de trabajo. Si alguna de ellas falla, todas las operaciones de la transacción se deben deshacer en conjunto mediante una operación denominada *rollback*. Este comportamiento garantiza que la base de datos nunca quede en un estado intermedio e inconsistente.

Por defecto, el módulo `sqlite3` agrupa dentro de una misma transacción las sentencias de modificación ejecutadas sobre una conexión. Los cambios que estas producen no se vuelven permanentes hasta que la transacción se confirma con el método `commit()`; si en su lugar se invoca `rollback()`, o si la conexión se cierra sin confirmar, el conjunto completo de cambios se descarta.

El siguiente ejemplo implementa una transferencia de saldo entre dos cuentas. Solo si las dos operaciones, restar el monto en el origen y sumarlo en el destino, se completan correctamente, se confirma la transacción; ante cualquier error, el conjunto se deshace:

```
import sqlite3

def transferir_saldo(cuenta_origen, cuenta_destino, monto):
    with sqlite3.connect('banco.db') as conexion:
        cursor = conexion.cursor()

        try:
            # Restar saldo de cuenta origen
            cursor.execute('''
                UPDATE cuentas
```

```
                SET saldo = saldo - ?
                WHERE numero = ?
            ''', (monto, cuenta_origen))

            # Sumar saldo a cuenta destino
            cursor.execute('''
                UPDATE cuentas
                SET saldo = saldo + ?
                WHERE numero = ?
            ''', (monto, cuenta_destino))

            # Confirmar transacción
            conexion.commit()
            print("Transferencia exitosa")

        except Exception as e:
            # Deshacer cambios en caso de error
            conexion.rollback()
            print(f"Error en la transferencia: {e}")
```

13.8 Manejo de errores

El manejo adecuado de los errores constituye una parte esencial del trabajo con bases de datos. El módulo `sqlite3` define una jerarquía de excepciones que permite distinguir con precisión el tipo de falla ocurrida; todas ellas derivan de `sqlite3.Error`. Al implementar el manejo de errores se recomienda capturar primero las subclases más específicas y, al final, `sqlite3.Error` como caso genérico:

```
import sqlite3

def insertar_persona_segura(documento, nombre, apellido, edad):
    try:
        with sqlite3.connect('personas.db') as conexion:
            cursor = conexion.cursor()

            cursor.execute('''
                INSERT INTO personas (documento, nombre, apellido, edad)
                VALUES (?, ?, ?, ?)
            ''', (documento, nombre, apellido, edad))

            conexion.commit()
            return True, "Persona insertada exitosamente"

    except sqlite3.IntegrityError as e:
        # Violación de restricción (ej: documento duplicado)
        return False, f"Error de integridad: {e}"

    except sqlite3.OperationalError as e:
        # Error operacional (ej: tabla no existe)
        return False, f"Error operacional: {e}"

    except sqlite3.Error as e:
        # Otros errores de SQLite
```

```
        return False, f"Error de base de datos: {e}"

# Uso
exito, mensaje = insertar_persona_segura('12345678', 'Juan', 'Pérez', 30)
print(mensaje)

# Intentar insertar el mismo documento nuevamente
exito, mensaje = insertar_persona_segura('12345678', 'Pedro', 'García', 25)
print(mensaje)  # Error: documento ya existe
```

13.9 Ejemplo completo: Sistema de gestión de personas

El siguiente ejemplo completo integra las operaciones CRUD (Create, Read, Update, Delete):

Sistema de gestión de personas con SQLite

```
"""
Sistema de gestión de personas con SQLite
Ejemplo completo de operaciones CRUD (Create, Read, Update, Delete)
"""

import sqlite3


def crear_tabla():
    """Crea la tabla personas si no existe"""
    with sqlite3.connect('personas.db') as conexion:
        cursor = conexion.cursor()
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS personas (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                documento TEXT NOT NULL UNIQUE,
                nombre TEXT NOT NULL,
                apellido TEXT NOT NULL,
                edad INTEGER CHECK(edad ≥ 0)
            )
        ''')
        conexion.commit()
        print("Tabla creada o verificada exitosamente")


def insertar_persona(documento, nombre, apellido, edad):
    """Inserta una nueva persona en la base de datos"""
    try:
        with sqlite3.connect('personas.db') as conexion:
            cursor = conexion.cursor()
            cursor.execute('''
                INSERT INTO personas (documento, nombre, apellido, edad)
                VALUES (?, ?, ?, ?)
            ''', (documento, nombre, apellido, edad))
            conexion.commit()
            return True, cursor.lastrowid
    except sqlite3.IntegrityError:
        return False, "El documento ya existe en la base de datos"
    except sqlite3.Error as e:
        return False, f"Error de base de datos: {e}"
```

```
def listar_todas():
    """Lista todas las personas de la base de datos"""
    with sqlite3.connect('personas.db') as conexion:
        conexion.row_factory = sqlite3.Row
        cursor = conexion.cursor()
        cursor.execute('SELECT * FROM personas ORDER BY apellido, nombre')
        return cursor.fetchall()


def buscar_por_documento(documento):
    """Busca una persona por su documento"""
    with sqlite3.connect('personas.db') as conexion:
        conexion.row_factory = sqlite3.Row
        cursor = conexion.cursor()
        cursor.execute('''
            SELECT * FROM personas WHERE documento = ?
        ''', (documento,))
        return cursor.fetchone()


def buscar_por_apellido(apellido):
    """Busca personas por apellido (búsqueda parcial)"""
    with sqlite3.connect('personas.db') as conexion:
        conexion.row_factory = sqlite3.Row
        cursor = conexion.cursor()
        cursor.execute('''
            SELECT * FROM personas
            WHERE apellido LIKE ?
            ORDER BY apellido, nombre
        ''', (f'%{apellido}%',))
        return cursor.fetchall()


def actualizar_edad(documento, nueva_edad):
    """Actualiza la edad de una persona"""
    with sqlite3.connect('personas.db') as conexion:
        cursor = conexion.cursor()
        cursor.execute('''
            UPDATE personas
            SET edad = ?
            WHERE documento = ?
        ''', (nueva_edad, documento))
        conexion.commit()
        return cursor.rowcount > 0


def eliminar_persona(documento):
    """Elimina una persona de la base de datos"""
    with sqlite3.connect('personas.db') as conexion:
        cursor = conexion.cursor()
        cursor.execute('''
            DELETE FROM personas
            WHERE documento = ?
        ''', (documento,))
        conexion.commit()
        return cursor.rowcount > 0


def mostrar_menu():
```

```
    """Muestra el menú de opciones"""
    print("\n" + "="*50)
    print("SISTEMA DE GESTIÓN DE PERSONAS")
    print("="*50)
    print("1. Insertar nueva persona")
    print("2. Listar todas las personas")
    print("3. Buscar por documento")
    print("4. Buscar por apellido")
    print("5. Actualizar edad")
    print("6. Eliminar persona")
    print("0. Salir")
    print("="*50)


def main():
    """Función principal del programa"""
    # Crear la tabla al iniciar
    crear_tabla()

    while True:
        mostrar_menu()
        opcion = input("\nSeleccione una opción: ")

        if opcion == "1":
            print("\n--- INSERTAR NUEVA PERSONA ---")
            documento = input("Documento: ")
            nombre = input("Nombre: ")
            apellido = input("Apellido: ")
            try:
                edad = int(input("Edad: "))
                exito, resultado = insertar_persona(documento, nombre, apellido, edad)
                if exito:
                    print(f"✓ Persona insertada con ID: {resultado}")
                else:
                    print(f"✗ Error: {resultado}")
            except ValueError:
                print("✗ Error: La edad debe ser un número entero")

        elif opcion == "2":
            print("\n--- LISTADO DE PERSONAS ---")
            personas = listar_todas()
            if personas:
                print(f"\nTotal: {len(personas)} persona(s)\n")
                for persona in personas:
                    print(f"ID: {persona['id']}")
                    print(f"  Documento: {persona['documento']}")
                    print(f"  Nombre: {persona['nombre']} {persona['apellido']}")
                    print(f"  Edad: {persona['edad']} años")
                    print()
            else:
                print("No hay personas registradas")

        elif opcion == "3":
            print("\n--- BUSCAR POR DOCUMENTO ---")
            documento = input("Ingrese el documento: ")
            persona = buscar_por_documento(documento)
            if persona:
                print(f"\nPersona encontrada:")
                print(f"  ID: {persona['id']}")
                print(f"  Nombre: {persona['nombre']} {persona['apellido']}")
                print(f"  Documento: {persona['documento']}")
```

```
                print(f"  Edad: {persona['edad']} años")
            else:
                print("No se encontró ninguna persona con ese documento")

        elif opcion == "4":
            print("\n--- BUSCAR POR APELLIDO ---")
            apellido = input("Ingrese el apellido (o parte): ")
            personas = buscar_por_apellido(apellido)
            if personas:
                print(f"\nSe encontraron {len(personas)} persona(s):\n")
                for persona in personas:
                    print(f"  {persona['nombre']} {persona['apellido']} "
                          f"(DNI: {persona['documento']}, {persona['edad']} años)")
            else:
                print("No se encontraron personas con ese apellido")

        elif opcion == "5":
            print("\n--- ACTUALIZAR EDAD ---")
            documento = input("Documento de la persona: ")
            try:
                nueva_edad = int(input("Nueva edad: "))
                if actualizar_edad(documento, nueva_edad):
                    print("✓ Edad actualizada correctamente")
                else:
                    print("✗ No se encontró ninguna persona con ese documento")
            except ValueError:
                print("✗ Error: La edad debe ser un número entero")

        elif opcion == "6":
            print("\n--- ELIMINAR PERSONA ---")
            documento = input("Documento de la persona a eliminar: ")
            confirmacion = input("¿Está seguro? (s/n): ")
            if confirmacion.lower() == 's':
                if eliminar_persona(documento):
                    print("✓ Persona eliminada correctamente")
                else:
                    print("✗ No se encontró ninguna persona con ese documento")
            else:
                print("Operación cancelada")

        elif opcion == "0":
            print("\n¡Hasta luego!")
            break

        else:
            print("\n✗ Opción inválida. Intente nuevamente.")


if __name__ == "__main__":
    main()
```

13.10 Patrón DAO (Data Access Object)

Una práctica habitual para organizar el acceso a la base de datos consiste en encapsular las operaciones CRUD en clases especializadas, denominadas *DAO* (*Data Access Object*). Este esquema reduce la duplicación de código, centraliza los detalles de conexión y facilita la realización de pruebas.

Una clase DAO concentra las operaciones de creación, consulta, actualización y eliminación sobre una tabla. Es habitual definir métodos como `crear()` para insertar registros, `obtener_por_id()` —o `obtener_por_documento()`, según el criterio de búsqueda— para recuperar una fila, `listar()` para obtener la totalidad de los registros, y `actualizar()` y `eliminar()` para modificar y suprimir filas respectivamente. Cada método encapsula la sentencia SQL correspondiente y expone una interfaz simple, de modo que el resto de la aplicación queda desacoplada de los detalles de acceso a los datos.

El siguiente ejemplo presenta una clase `PersonaDAO` que concentra las operaciones de creación, consulta, actualización y eliminación sobre la tabla `personas`:

```
import sqlite3

class PersonaDAO:
    def __init__(self, db_path='personas.db'):
        self.db_path = db_path

    def _conn(self):
        return sqlite3.connect(self.db_path)

    def crear(self, documento, nombre, apellido, edad=None):
        with self._conn() as cx:
            cx.execute('''
                INSERT INTO personas (documento, nombre, apellido, edad)
                VALUES (?, ?, ?, ?)
            ''', (documento, nombre, apellido, edad))

    def obtener_por_documento(self, documento):
        with self._conn() as cx:
            cx.row_factory = sqlite3.Row
            cur = cx.execute('SELECT * FROM personas WHERE documento = ?',
            ↪  (documento,))
            return cur.fetchone()

    def actualizar_edad(self, documento, edad):
        with self._conn() as cx:
            cur = cx.execute('UPDATE personas SET edad=? WHERE documento=?', (edad,
            ↪  documento))
            return cur.rowcount

    def eliminar(self, documento):
        with self._conn() as cx:
            cur = cx.execute('DELETE FROM personas WHERE documento=?', (documento,))
            return cur.rowcount
```

Cuando la base de datos posee más de una tabla, la práctica habitual consiste en definir un DAO por cada tabla principal, es decir, por cada entidad representada en el esquema. Así, en una base `ventas.db` que contiene las tablas `clientes` y `pedidos`, se definen las clases `ClienteDAO` y `PedidoDAO`; las consultas de cada

tabla quedan así aisladas en la clase a la que corresponden.

Las relaciones de uno a muchos se resuelven por medio de las claves foráneas. En la relación que vincula a los clientes con sus pedidos, un cliente posee muchos pedidos; la tabla `pedidos` contiene la columna `cliente_id` que referencia a un registro de `clientes`. El DAO de la tabla que contiene la clave foránea recibe el identificador de la fila relacionada en sus operaciones de creación y ofrece métodos de consulta que toman ese identificador como criterio, como `obtener_por_cliente()`:

```
import sqlite3

class PedidoDAO:
    def __init__(self, db_path='ventas.db'):
        self.db_path = db_path

    def _conn(self):
        return sqlite3.connect(self.db_path)

    def crear(self, cliente_id, fecha, total):
        with self._conn() as cx:
            cur = cx.execute('''
                INSERT INTO pedidos (cliente_id, fecha, total)
                VALUES (?, ?, ?)
            ''', (cliente_id, fecha, total))
            return cur.lastrowid

    def obtener_por_cliente(self, cliente_id):
        with self._conn() as cx:
            cx.row_factory = sqlite3.Row
            cur = cx.execute('''
                SELECT * FROM pedidos WHERE cliente_id = ?
            ''', (cliente_id,))
            return cur.fetchall()
```

Para las consultas que combinan información de ambas tablas, como obtener el nombre del cliente junto con cada uno de sus pedidos, se emplean sentencias `JOIN` que integran los datos de las tablas relacionadas; estas consultas pueden incorporarse como métodos del DAO de la misma manera que las anteriores.

Las relaciones de muchos a muchos no pueden representarse con una sola clave foránea: se resuelven introduciendo una tabla intermedia que vincula las dos tablas principales. Cuando esa tabla contiene únicamente las claves foráneas a ambas tablas, que en conjunto forman su clave primaria, su único propósito es registrar las asociaciones entre los registros. Por ejemplo, si un pedido puede incluir varios artículos y un artículo puede pertenecer a varios pedidos, se define una tabla `pedido_articulo` con las columnas

`pedido_id` y `articulo_id`. Las operaciones de esta tabla, como `agregar_articulo(pedido_id, articulo_id)`, `quitar_articulo(pedido_id, articulo_id)` u `obtener_articulos_del_pedido(pedido_id)`, pueden ubicarse en el DAO de una de las tablas principales o en un DAO dedicado a la tabla de unión.

El segundo caso se presenta cuando la tabla intermedia, además de las claves foráneas, almacena datos propios; en esa situación deja de ser una simple tabla de unión y pasa a constituir una entidad del dominio. Es lo que ocurre con `detalle_pedido`, tabla intermedia entre `pedidos` y `articulos`: cada una de sus filas corresponde a un artículo incluido en un pedido y, junto a las claves `pedido_id` y `articulo_id`, conserva información propia como la cantidad comprada o el precio unitario aplicado. Por tratarse de una entidad, la tabla recibe su propio DAO, `DetallePedidoDAO`, con las operaciones completas de creación, consulta, actualización y eliminación: su método `crear()` recibe los identificadores de ambos extremos junto con los datos propios del detalle, y un método como `obtener_por_pedido(pedido_id)` recupera todos los detalles que componen un pedido. Tanto en uno como en otro caso, las consultas que integran información de las tablas relacionadas se componen mediante `JOIN` y pueden incorporarse como métodos de los DAO involucrados.



14.1 Introducción a los ORM

Los *ORM* por (*Object-Relational Mapping*), constituyen una herramienta ampliamente utilizada en la programación, todos los lenguajes incluyen al menos una librería con esta funcionalidad, tales como *JPA* en Java y con *Sequelize* en JavaScript.

La existencia de los ORM se explica por el **desajuste** (*impedance mismatch*) que se produce entre dos modelos de representación de la información: el modelo relacional, organizado en tablas, filas y claves, y el modelo orientado a objetos, organizado en clases, objetos y relaciones. La brecha entre ambos es doble. Por un lado, existe una **brecha sintáctica**: el modelo relacional se manipula mediante SQL, un lenguaje declarativo cuyas sentencias deben escribirse y gestionarse en forma de cadenas de texto, mientras que el modelo orientado a objetos se manipula mediante llamadas a métodos sobre objetos. Por el otro, subsiste una **brecha semántica**: una fila no es equivalente a un objeto, ya que la fila no posee identidad más allá de sus datos, en tanto que el objeto combina identidad, estado y comportamiento; del mismo modo, una tabla no es una clase.

Un ORM automatiza la traducción entre ambos mundos: empareja tablas en clases, filas en objetos, columnas en atributos y transacciones en sesiones de trabajo. Al hacerlo, reduce de manera significativa la brecha sintáctica y semántica, y permite que el código de persistencia se exprese en los términos del modelo orientado a objetos.

14.2 SQLAlchemy

SQLAlchemy es el ORM más completo y utilizado del ecosistema de Python. Proporciona dos enfoques complementarios para trabajar con bases de datos:

- **SQLAlchemy Core**: Una capa de abstracción SQL que permite construir consultas de forma programática
- **SQLAlchemy ORM**: El ORM propiamente dicho, que permite trabajar con objetos Python

En este capítulo se trabajará exclusivamente con SQLAlchemy ORM, que es el enfoque más utilizado para aplicaciones orientadas a objetos.

14.2.1 Instalación

SQLAlchemy no forma parte de la biblioteca estándar de Python, por lo que debe instalarse previamente mediante el administrador de paquetes `pip`:

```
pip install sqlalchemy
```

14.3 Configuración básica

Para comenzar a trabajar con SQLAlchemy es necesario realizar tres pasos fundamentales:

1. Crear un motor de base de datos (*Engine*)
2. Declarar una clase base para los modelos
3. Crear una sesión para interactuar con la base de datos

```
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

# 1. Crear el motor de base de datos
engine = create_engine('sqlite:///personas.db', echo=True)

# 2. Crear la clase base para los modelos
Base = declarative_base()

# 3. Crear una fábrica de sesiones
Session = sessionmaker(bind=engine)
```

El parámetro `echo=True` hace que SQLAlchemy imprima en consola todas las consultas SQL que ejecuta, lo cual resulta de gran utilidad durante la depuración.

14.3.1 Arquitectura y componentes principales

La capa ORM de SQLAlchemy se apoya en una serie de componentes bien diferenciados. Comprender el rol de cada uno de ellos facilita el diseño y la depuración de las aplicaciones de persistencia:

Engine Representa la *puerta de entrada* a la base de datos. Gestiona el *pool* de conexiones y sabe cómo emitir instrucciones SQL a un *DBAPI driver*. No mantiene estado de objetos de negocio.

Dialect Capa que traduce construcciones genéricas de SQLAlchemy a la sintaxis específica del motor (SQLite, PostgreSQL, etc.). Se selecciona implícitamente según la URL (por ejemplo, `sqlite:///` …).

MetaData Colección de objetos que describen esquemas de tablas (nombres, columnas, claves, índices). Se accede vía `Base.metadata` cuando se usa el estilo declarativo.

Declarative Base Clase base desde la que heredan los modelos. Conecta la definición de atributos de clase con las columnas de la tabla y registra la tabla dentro de los metadatos.

Session Unidad de trabajo (*Unit of Work*). Acumula cambios sobre objetos (nuevos, modificados, eliminados) y coordina el envío de dichas mutaciones a la base de datos dentro de una transacción.

Mapper Objeto interno que enlaza una clase Python con su tabla y reglas de persistencia. Se crea automáticamente al declarar la clase.

14.3.2 Ciclo de vida y estados de los objetos

Las instancias gestionadas por la sesión atraviesan distintos **estados** a lo largo de su ciclo de vida:

- **Transient** (transitorio): Objeto recién creado en Python, aún no asociado a una sesión ni a ninguna identidad en la base de datos.

- **Pending**: Tras ejecutar `session.add(obj)` el objeto queda en cola para inserción; todavía no se emitió SQL de INSERT (salvo que ocurra un `flush` anticipado).
- **Persistent**: Luego de un `flush` (implícito o explícito) el objeto obtiene identidad (clave primaria) y está sincronizado con la fila en la base de datos.
- **Dirty**: Subestado de *Persistent*; la sesión detectó cambios en atributos y los enviará al próximo `flush`.
- **Detached**: El objeto ya no está asociado a ninguna sesión (por cierre de sesión o expulsión explícita con `session.expunge(obj)`). Puede leerse pero los cambios no se rastrean hasta volver a asociarlo con `session.merge(obj)`.
- **Deleted**: Marcado para eliminación (`session.delete(obj)`). La eliminación física se concreta al `flush` / `commit`.

Flush vs Commit

- **Flush**: Sincroniza el estado pendiente de la sesión con la base de datos emitiendo INSERT/UPDATE/DELETE dentro de la transacción abierta. No confirma la transacción.
- **Commit**: Finaliza (confirma) la transacción. Antes ejecuta un flush implícito si hay cambios pendientes.

Un `rollback()` revierte la transacción en curso: los objetos *pending* vuelven al estado *transient* y las modificaciones *dirty* se descartan.

14.3.3 Patrón Unidad de Trabajo (Unit of Work)

La sesión mantiene, además, un registro interno de identidades (*identity map*) que garantiza que, para una misma clave primaria, exista una única instancia viva durante el ciclo de trabajo. Esto evita inconsistencias y facilita el rastreo de los cambios diferenciales al momento de emitir SQL.

14.3.4 Sintaxis habitual de creación del Engine

A continuación se presentan los formatos de URL más habituales junto con algunos de los parámetros más frecuentes de `create_engine()`:

```python
from sqlalchemy import create_engine

# Formatos de URL comunes
# SQLite (archivo local)
create_engine('sqlite:///app.db')
# Memoria (solo duración del proceso)
create_engine('sqlite:///:memory:')
# PostgreSQL (requiere driver psycopg2 instalado)
create_engine('postgresql+psycopg2://usuario:clave@localhost:5432/base')
# MySQL (driver mysqlclient o pymysql)
create_engine('mysql+pymysql://usuario:clave@localhost:3306/base?charset=utf8mb4')

# Parámetros frecuentes
create_engine(
    'sqlite:///app.db',
    echo=False,          # Mostrar o no SQL
    future=True,         # Estilo 2.x (select() explícito)
    pool_pre_ping=True   # Verificar conexiones antes de usar
)
```

14.3.5 Creación de sesiones: patrones de uso

Se recomienda crear una *fábrica* de sesiones con `sessionmaker` y obtener, a partir de ella, instancias de sesión de corta duración (una por transacción, petición web o caso de uso):

```python
from sqlalchemy.orm import sessionmaker

SessionLocal = sessionmaker(bind=engine, autoflush=True, expire_on_commit=True)

# Obtención y uso mínimo
session = SessionLocal()
try:
    # Operaciones
    ...
    session.commit()
except Exception:
    session.rollback()
    raise
finally:
    session.close()
```

Parámetros relevantes de sessionmaker

- `autoflush` (True por defecto): Si está activo, se ejecuta un flush automático antes de ciertas consultas para garantizar coherencia.
- `expire_on_commit` (True por defecto): Al confirmar la transacción, los atributos cargados se marcan como expirados; un nuevo acceso re-consulta datos para asegurar frescura.

14.3.6 Resumen de métodos clave de Session

add(obj) Coloca un objeto en estado pending.

add_all(lista) Coloca una colección de objetos en estado pending.

commit() Confirma la transacción (incluye flush implícito).

rollback() Revierte cambios desde el último commit.

flush() Fuerza sincronización sin cerrar la transacción.

delete(obj) Marca para eliminación.

merge(obj) Reconcilia un objeto detached con la sesión, devolviendo la instancia gestionada.

refresh(obj) Vuelve a consultar la fila y actualiza atributos.

expunge(obj) Saca un objeto de la sesión (pasa a detached).

close() Cierra la sesión y su transacción implícita si está activa.

14.4 Definición de modelos

En SQLAlchemy, cada tabla de la base de datos se representa mediante una clase Python que hereda de la clase `Base` definida en la sección anterior. Los atributos de la clase representan las columnas de la tabla, y en ella también pueden incorporarse los métodos que implementan la lógica de negocio de la entidad:

```
from sqlalchemy import Column, Integer, String

class Persona(Base):
    __tablename__ = 'personas'

    id = Column(Integer, primary_key=True, autoincrement=True)
    documento = Column(String(20), nullable=False, unique=True)
    nombre = Column(String(100), nullable=False)
    apellido = Column(String(100), nullable=False)
    edad = Column(Integer)

    def __repr__(self):
        return f"<Persona(nombre='{self.nombre}', apellido='{self.apellido}')>"

    def __str__(self):
        return f"{self.nombre} {self.apellido} (DNI: {self.documento})"
```

14.4.1 Tipos de datos en SQLAlchemy

SQLAlchemy proporciona un conjunto de tipos de datos que se mapean automáticamente a los tipos nativos de cada motor de base de datos:

Tipo SQLAlchemy	Descripción
`Integer`	Números enteros
`String(length)`	Cadenas de longitud fija o variable
`Text`	Cadenas de texto largo sin límite de longitud
`Float`	Números de punto flotante
`Boolean`	Valores booleanos
`Date`	Fechas (año, mes, día)
`DateTime`	Fechas con hora
`Time`	Hora del día
`LargeBinary`	Datos binarios (BLOB)

14.4.2 Restricciones de columna

Las restricciones se especifican como parámetros al momento de declarar cada columna:

- `primary_key=True`: Define la columna como clave primaria
- `nullable=False`: La columna no puede contener valores NULL
- `unique=True`: Todos los valores deben ser únicos
- `default=valor`: Valor por defecto
- `autoincrement=True`: Genera valores automáticamente

14.5 Creación de tablas

Una vez definidos los modelos, es necesario crear las tablas correspondientes en la base de datos. En SQLAlchemy esta tarea se resuelve con una única instrucción:

```
# Crear todas las tablas definidas
Base.metadata.create_all(engine)
```

Este comando examina todos los modelos que heredan de `Base` y crea las tablas correspondientes en el caso de que aún no existan.

14.6 Operaciones CRUD con SQLAlchemy

Esta sección describe la sintaxis fundamental de las operaciones de persistencia siguiendo el patrón de Unidad de Trabajo presentado anteriormente. Cada operación modifica el estado de los objetos en memoria y difiere su sincronización física con la base de datos hasta que se produce un flush o un commit.

14.6.1 Resumen rápido de sintaxis de consulta

SQLAlchemy ofrece dos estilos de construcción de consultas, que coexisten en la versión actual de la librería. El estilo clásico, ofrecido des de la primera versión de la librería, se basa en `session.query()`; el estilo moderno, formalizado en versiones posteriores, se basa en el objeto `select()`. En este capítulo se utiliza el estilo clásico pero conviene conocer ambos: el primero sigue presente en gran parte del código existente, mientras que el segundo constituye el camino recomendado para los desarrollos nuevos.

El estilo clásico modela la consulta como una cadena de llamadas a métodos. La expresión `session.query(Persona)` indica la entidad cuyas filas se desean consultar; a partir de ella, cada método agrega una cláusula y devuelve un nuevo objeto sobre el cual se encadenan las llamadas siguientes: `filter()` establece las condiciones, `order_by()` el ordenamiento y `limit()` la cantidad de filas. La traducción a SQL ocurre recién cuando se invoca un método terminal, como `all()`, `first()` o `count()`; antes de ese momento la consulta es un valor en construcción, no una ejecución. Los criterios se expresan mediante operadores de comparación aplicados sobre los atributos de la clase, como `Persona.apellido == 'Pérez'`, que el ORM sobrecarga para representar condiciones SQL:

```
session.query(Persona).filter(Persona.apellido ==
↪  'Pérez').order_by(Persona.nombre).limit(10)
```

El estilo 2.x modifica ese esquema al separar la construcción de la consulta de su ejecución. La consulta se representa como un objeto `select()` independiente, que se asigna a una variable y sobre el cual se agregan las cláusulas con los métodos `where()`, `order_by()` y `limit()`. La ejecución se delega a la sesión mediante `session.execute()`, que devuelve un objeto `Result` del cual se extraen las entidades resultantes con `result.scalars().all()`. La ventaja de este esquema es que la consulta constituye un valor reutilizable y componible: puede asignar-

se, pasarse como argumento a una función o combinarse con otras antes de decidir cómo se ejecuta, en lugar de quedar atada a una cadena de llamadas:

```
from sqlalchemy import select

stmt = select(Persona).where(Persona.apellido ==
↪  'Pérez').order_by(Persona.nombre).limit(10)
result = session.execute(stmt)
personas = result.scalars().all()
```

Cuando una condición combina varias cláusulas, estas se componen mediante los operadores booleanos `and_()`, `or_()` y `not_()`, que reciben las condiciones como argumentos y pueden anidarse para expresar filtros complejos dentro de `filter()`:

```
from sqlalchemy import and_, or_, not_

session.query(Persona).filter(
    and_(
        Persona.edad ≥ 18,
        or_(Persona.apellido.like('P%'), Persona.apellido.like('G%')),
        not_(Persona.nombre == 'Test')
    )
).all()
```

Las cláusulas de comparación pueden utilizar los operadores `==`, `≠`, `>`, `≥`, `<`, `≤`, `like()`, `in_([…])`, `between(a,b)`, `is_(None)` y `isnot(None)`.

14.6.2 Inserción de filas

Para insertar una nueva fila se crea una instancia del modelo y se la agrega a la sesión mediante `session.add()`:

```
# Crear una sesión
session = Session()

# Crear una nueva persona
persona = Persona(
    documento='12345678',
    nombre='Juan',
    apellido='Pérez',
    edad=30
)

# Agregar a la sesión
session.add(persona)

# Confirmar los cambios
```

```
session.commit()

print(f"Persona insertada con ID: {persona.id}")

# Cerrar la sesión
session.close()
```

Para insertar varias filas en una misma operación se utiliza `session.add_all()`:

```
session = Session()

personas = [
    Persona(documento='11111111', nombre='María', apellido='González', edad=25),
    Persona(documento='22222222', nombre='Pedro', apellido='Martínez', edad=35),
    Persona(documento='33333333', nombre='Ana', apellido='López', edad=28)
]

session.add_all(personas)
session.commit()
session.close()
```

14.6.3 Consultas

Para obtener todas las filas de una tabla se ejecuta la consulta con `session.query(Persona).all()`; el método `all()` materializa el resultado y devuelve una lista de instancias del modelo, que se recorren como cualquier colección de objetos:

```
session = Session()

# Obtener todas las personas
personas = session.query(Persona).all()

for persona in personas:
    print(persona)

session.close()
```

Las condiciones de filtrado se incorporan con `filter_by()`, que recibe los valores buscados por nombre de columna, o con `filter()`, que admite condiciones más generales construidas con los atributos de la clase y los operadores de comparación mencionados en la sección anterior. El método `first()` ejecuta la consulta y devuelve la primera fila, o `None` si ninguna la satisface:

```
session = Session()

# Buscar por documento
persona = session.query(Persona).filter_by(documento='12345678').first()
```

```
print(persona)

# Buscar personas mayores de 25 años
mayores = session.query(Persona).filter(Persona.edad > 25).all()
for persona in mayores:
    print(persona)

# Buscar con LIKE
apellido_buscar = 'Pér%'
personas = session.query(Persona).filter(
    Persona.apellido.like(apellido_buscar)
).all()

session.close()
```

El ordenamiento se agrega con `order_by()`, que recibe una o más columnas en el orden de prioridad deseado; aplicar `desc()` sobre una columna indica que el orden es descendente:

```
session = Session()

# Ordenar por apellido y luego por nombre
personas = session.query(Persona).order_by(
    Persona.apellido,
    Persona.nombre
).all()

# Ordenar de forma descendente
personas_desc = session.query(Persona).order_by(
    Persona.edad.desc()
).all()

session.close()
```

La cantidad de filas devueltas se restringe con `limit()`, mientras que `get()` recupera directamente una fila a partir de su clave primaria:

```
session = Session()

# Obtener las primeras 5 filas
primeros_cinco = session.query(Persona).limit(5).all()

# Obtener una fila específica por ID
persona = session.query(Persona).get(1)  # ID = 1

session.close()
```

14.6.4 Actualización de filas

Para actualizar una fila, primero se la busca mediante una consulta, luego se modifican sus atributos y finalmente se confirman los cambios:

```
session = Session()

# Buscar la persona
persona = session.query(Persona).filter_by(documento='12345678').first()

if persona:
    # Modificar atributos
    persona.edad = 31

    # Confirmar cambios
    session.commit()
    print("Persona actualizada")
else:
    print("Persona no encontrada")

session.close()
```

Además de la actualización objeto por objeto, es posible efectuar actualizaciones masivas que modifican varias filas a la vez:

```
session = Session()

# Incrementar la edad de todas las personas
session.query(Persona).update({Persona.edad: Persona.edad + 1})
session.commit()

session.close()
```

14.6.5 Eliminación de filas

Para eliminar una fila, se la busca y se la marca para eliminación con `session.delete()`:

```
session = Session()

# Buscar la persona
persona = session.query(Persona).filter_by(documento='12345678').first()

if persona:
    # Eliminar
    session.delete(persona)
    session.commit()
    print("Persona eliminada")
else:
    print("Persona no encontrada")

session.close()
```

La eliminación masiva se efectúa a partir de una consulta:

```
session = Session()

# Eliminar todas las personas menores de 18 años
session.query(Persona).filter(Persona.edad < 18).delete()
session.commit()

session.close()
```

14.7 Context managers y gestión de sesiones

Para garantizar que las sesiones se cierren correctamente, es recomendable encapsular su ciclo de vida en un context manager que confirme los cambios cuando la operación se completa, los revierta ante un error y cierre la sesión en todos los casos:

```
from contextlib import contextmanager

@contextmanager
def obtener_session():
    session = Session()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()

# Uso
with obtener_session() as session:
    persona = Persona(
        documento='44444444',
        nombre='Luis',
        apellido='Rodríguez',
        edad=42
    )
    session.add(persona)
```

14.8 Relaciones entre tablas

Una de las características más potentes de SQLAlchemy es el manejo de relaciones entre tablas, que permite recorrer los vínculos del modelo en ambos sentidos sin escribir consultas SQL.

14.8.1 Relación uno a muchos (One-to-Many)

El caso más habitual es la relación uno a muchos. Por ejemplo, un cliente puede tener múltiples órdenes de compra:

```python
from sqlalchemy import ForeignKey
from sqlalchemy.orm import relationship

class Cliente(Base):
    __tablename__ = 'clientes'

    id = Column(Integer, primary_key=True)
    nombre = Column(String(100), nullable=False)
    email = Column(String(100), unique=True)

    # Relación uno a muchos
    ordenes = relationship('Orden', back_populates='cliente')

    def __repr__(self):
        return f"<Cliente(nombre='{self.nombre}')>"

class Orden(Base):
    __tablename__ = 'ordenes'

    id = Column(Integer, primary_key=True)
    numero = Column(String(50), unique=True)
    total = Column(Float)
    cliente_id = Column(Integer, ForeignKey('clientes.id'))

    # Relación muchos a uno
    cliente = relationship('Cliente', back_populates='ordenes')

    def __repr__(self):
        return f"<Orden(numero='{self.numero}', total={self.total})>"
```

Uso de la relación:

```python
# Crear tablas
Base.metadata.create_all(engine)

with obtener_session() as session:
    # Crear un cliente
    cliente = Cliente(nombre='Juan Pérez', email='juan@example.com')

    # Crear órdenes para el cliente
    orden1 = Orden(numero='ORD-001', total=150.50)
    orden2 = Orden(numero='ORD-002', total=200.00)

    # Asociar órdenes al cliente
    cliente.ordenes.append(orden1)
    cliente.ordenes.append(orden2)

    session.add(cliente)

# Consultar
with obtener_session() as session:
    cliente = session.query(Cliente).filter_by(nombre='Juan Pérez').first()

    print(f"Cliente: {cliente.nombre}")
    print(f"Órdenes:")
    for orden in cliente.ordenes:
        print(f"  - {orden.numero}: ${orden.total}")
```

14.8.2 Relación muchos a uno (Many-to-One)

La relación muchos a uno constituye, simplemente, el lado inverso de la relación uno a muchos. En el ejemplo precedente, varias órdenes pertenecen a un único cliente; acceder a la relación desde el lado de la orden permite obtener el cliente asociado:

```
with obtener_session() as session:
    orden = session.query(Orden).filter_by(numero='ORD-001').first()

    print(f"Orden: {orden.numero}")
    print(f"Cliente: {orden.cliente.nombre}")
    print(f"Email: {orden.cliente.email}")
```

14.8.3 Relación muchos a muchos (Many-to-Many)

Sin atributos adicionales en la tabla intermedia

Cuando un objeto puede vincularse con varios objetos del otro lado y viceversa, se recurre a una relación muchos a muchos mediada por una tabla de asociación. Por ejemplo, un estudiante puede inscribirse en múltiples cursos, y un curso puede contar con múltiples estudiantes:

```
from sqlalchemy import Table

# Tabla de asociación (intermedia)
estudiante_curso = Table(
    'estudiante_curso',
    Base.metadata,
    Column('estudiante_id', Integer, ForeignKey('estudiantes.id')),
    Column('curso_id', Integer, ForeignKey('cursos.id'))
)

class Estudiante(Base):
    __tablename__ = 'estudiantes'

    id = Column(Integer, primary_key=True)
    nombre = Column(String(100), nullable=False)
    matricula = Column(String(20), unique=True)

    # Relación muchos a muchos
    cursos = relationship(
        'Curso',
        secondary=estudiante_curso,
        back_populates='estudiantes'
    )

    def __repr__(self):
        return f"<Estudiante(nombre='{self.nombre}')>"

class Curso(Base):
```

```
    __tablename__ = 'cursos'

    id = Column(Integer, primary_key=True)
    codigo = Column(String(20), unique=True)
    nombre = Column(String(100), nullable=False)
    creditos = Column(Integer)

    # Relación muchos a muchos
    estudiantes = relationship(
        'Estudiante',
        secondary=estudiante_curso,
        back_populates='cursos'
    )

    def __repr__(self):
        return f"<Curso(codigo='{self.codigo}', nombre='{self.nombre}')>"
```

Uso:

```
# Crear tablas
Base.metadata.create_all(engine)

with obtener_session() as session:
    # Crear estudiantes
    est1 = Estudiante(nombre='Ana García', matricula='EST-001')
    est2 = Estudiante(nombre='Carlos López', matricula='EST-002')

    # Crear cursos
    curso1 = Curso(codigo='POO-101', nombre='Programación Orientada a Objetos',
    ↪  creditos=6)
    curso2 = Curso(codigo='BD-201', nombre='Bases de Datos', creditos=4)
    curso3 = Curso(codigo='WEB-301', nombre='Desarrollo Web', creditos=5)

    # Inscribir estudiantes en cursos
    est1.cursos.extend([curso1, curso2])
    est2.cursos.extend([curso1, curso3])

    session.add_all([est1, est2, curso1, curso2, curso3])

# Consultar inscripciones
with obtener_session() as session:
    estudiante = session.query(Estudiante).filter_by(matricula='EST-001').first()

    print(f"Estudiante: {estudiante.nombre}")
    print(f"Cursos inscritos:")
    for curso in estudiante.cursos:
        print(f"  - {curso.codigo}: {curso.nombre} ({curso.creditos} créditos)")

    # Desde el lado del curso
    curso = session.query(Curso).filter_by(codigo='POO-101').first()
    print(f"\nCurso: {curso.nombre}")
    print(f"Estudiantes inscritos:")
    for est in curso.estudiantes:
        print(f"  - {est.nombre} ({est.matricula})")
```

Con atributos adicionales en la tabla intermedia

Cuando la tabla intermedia necesita almacenar información adicional (por ejemplo, la nota obtenida por el estudiante en el curso o la fecha de inscripción), una tabla de asociación simple ya no resulta suficiente. En ese caso debe modelarse una **clase de asociación** (*Association Object*) explícita, con una clave primaria compuesta y relaciones muchos a uno hacia las entidades que vincula:

```
from sqlalchemy import Date
from datetime import date

class Inscripcion(Base):
    __tablename__ = 'inscripciones'

    estudiante_id = Column(Integer, ForeignKey('estudiantes.id'), primary_key=True)
    curso_id = Column(Integer, ForeignKey('cursos.id'), primary_key=True)
    fecha_inscripcion = Column(Date, default=date.today)
    nota_final = Column(Float)
    estado = Column(String(20))  # 'cursando', 'aprobado', 'desaprobado'

    # Relaciones
    estudiante = relationship('Estudiante', back_populates='inscripciones')
    curso = relationship('Curso', back_populates='inscripciones')

    def __repr__(self):
        return f"<Inscripcion(estudiante_id={self.estudiante_id},
        ↪  curso_id={self.curso_id})>"

class Estudiante(Base):
    __tablename__ = 'estudiantes'

    id = Column(Integer, primary_key=True)
    nombre = Column(String(100), nullable=False)
    matricula = Column(String(20), unique=True)

    # Relación con la tabla de asociación
    inscripciones = relationship('Inscripcion', back_populates='estudiante')

    def __repr__(self):
        return f"<Estudiante(nombre='{self.nombre}')>"

class Curso(Base):
    __tablename__ = 'cursos'

    id = Column(Integer, primary_key=True)
    codigo = Column(String(20), unique=True)
    nombre = Column(String(100), nullable=False)
    creditos = Column(Integer)

    # Relación con la tabla de asociación
    inscripciones = relationship('Inscripcion', back_populates='curso')

    def __repr__(self):
        return f"<Curso(codigo='{self.codigo}', nombre='{self.nombre}')>"
```

El siguiente fragmento crea inscripciones junto con sus datos adicionales y muestra cómo consultar la información asociada:

```
# Crear tablas
Base.metadata.create_all(engine)

with obtener_session() as session:
    # Crear estudiante y curso
    estudiante = Estudiante(nombre='María Rodríguez', matricula='EST-003')
    curso = Curso(codigo='ALG-101', nombre='Algoritmos', creditos=6)

    # Crear inscripción con datos adicionales
    inscripcion = Inscripcion(
        estudiante=estudiante,
        curso=curso,
        fecha_inscripcion=date(2025, 3, 1),
        estado='cursando'
    )

    session.add_all([estudiante, curso, inscripcion])

# Consultar con datos de inscripción
with obtener_session() as session:
    estudiante = session.query(Estudiante).filter_by(matricula='EST-003').first()

    print(f"Estudiante: {estudiante.nombre}")
    print(f"Inscripciones:")
    for insc in estudiante.inscripciones:
        print(f"  - {insc.curso.nombre}")
        print(f"    Fecha: {insc.fecha_inscripcion}")
        print(f"    Estado: {insc.estado}")
        if insc.nota_final:
            print(f"    Nota: {insc.nota_final}")

# Actualizar nota
with obtener_session() as session:
    inscripcion = session.query(Inscripcion).filter_by(
        estudiante_id=1,
        curso_id=1
    ).first()

    inscripcion.nota_final = 8.5
    inscripcion.estado = 'aprobado'
```

14.8.4 Parámetros importantes de relationship

back_populates Define el atributo complementario en la otra clase para mantener sincronía bidireccional explícita.

backref Crea automáticamente el atributo inverso (atajo sintáctico). Útil para reducir código cuando no se requieren configuraciones diferenciadas en cada lado.

cascade Controla cómo se propagan operaciones (`save-update`, `delete`, `delete-orphan`, etc.). Ejemplo: `cascade="all, delete-orphan"` en una relación uno-a-muchos que

posee semántica de composición fuerte.

lazy Estrategia de carga: `'select'` (por defecto), `'joined'` (JOIN inmediato), `'subquery'`, `'selectin'` (carga por lote eficiente), `'raise'` (lanza error si se accede sin haberla cargado), `'noload'`.

uselist Booleano que indica si el atributo representa una colección (True) o una única entidad (False). Útil para relaciones uno-a-uno.

secondary Tabla de asociación utilizada para relaciones muchos-a-muchos sin entidad intermedia explícita.

secondaryjoin Condición adicional para unir la tabla secundaria en relaciones más complejas (por ejemplo, claves compuestas o auto-relaciones).

order_by Orden por defecto de la colección (ej.: `order_by="Orden.fecha.desc()"`).

passive_deletes Optimiza eliminaciones delegándolas a la base (requiere ON DELETE CASCADE en la clave foránea).

14.8.5 Elección del patrón de muchos-a-muchos

- **Tabla de asociación simple**: Cuando solo se necesitan claves foráneas sin atributos adicionales.
- **Objeto de asociación (Association Object)**: Cuando se requieren campos extra (fecha, estado, métricas); se modela como clase y se define dos relaciones muchos-a-uno.

14.9 Ejemplo completo: Sistema de biblioteca

El siguiente ejemplo integra todos los conceptos vistos en el capítulo:

Sistema de biblioteca con SQLAlchemy

```
"""
Sistema de gestión de biblioteca usando SQLAlchemy ORM.
Demuestra relaciones uno a muchos y muchos a muchos con datos adicionales.
"""

from sqlalchemy import create_engine, Column, Integer, String, Date, ForeignKey, Table
from sqlalchemy.orm import declarative_base, relationship, sessionmaker
from contextlib import contextmanager
from datetime import date, timedelta

# Configuración de la base de datos
engine = create_engine('sqlite:///biblioteca.db', echo=False)
Base = declarative_base()
```

```
# expire_on_commit=False: los objetos devueltos por los DAO mantienen sus
# atributos cargados legibles al cerrarse la sesión. Las relaciones lazy,
# en cambio, solo pueden consultarse con la sesión abierta.
Session = sessionmaker(bind=engine, expire_on_commit=False)


@contextmanager
def obtener_session():
    """Context manager para gestionar sesiones de SQLAlchemy."""
    session = Session()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()


# Tabla de asociación muchos a muchos: libros y autores
libro_autor = Table(
    'libro_autor',
    Base.metadata,
    Column('libro_id', Integer, ForeignKey('libros.id')),
    Column('autor_id', Integer, ForeignKey('autores.id'))
)


class Autor(Base):
    """Representa un autor de libros."""
    __tablename__ = 'autores'

    id = Column(Integer, primary_key=True)
    nombre = Column(String(100), nullable=False)
    apellido = Column(String(100), nullable=False)
    nacionalidad = Column(String(50))

    # Relación muchos a muchos con libros
    libros = relationship('Libro', secondary=libro_autor, back_populates='autores')

    def __repr__(self):
        return f"<Autor(nombre='{self.nombre} {self.apellido}')>"

    def __str__(self):
        return f"{self.nombre} {self.apellido}"


class Libro(Base):
    """Representa un libro en la biblioteca."""
    __tablename__ = 'libros'

    id = Column(Integer, primary_key=True)
    isbn = Column(String(20), unique=True, nullable=False)
    titulo = Column(String(200), nullable=False)
    editorial = Column(String(100))
    anio = Column(Integer)
    ejemplares_totales = Column(Integer, default=1)
    ejemplares_disponibles = Column(Integer, default=1)

    # Relaciones
```

```
    autores = relationship('Autor', secondary=libro_autor, back_populates='libros')
    prestamos = relationship('Prestamo', back_populates='libro')

    def __repr__(self):
        return f"<Libro(titulo='{self.titulo}')>"

    def __str__(self):
        autores_str = ', '.join([str(autor) for autor in self.autores])
        return f"{self.titulo} - {autores_str} ({self.anio})"

    @property
    def disponible(self):
        """Verifica si hay ejemplares disponibles."""
        return self.ejemplares_disponibles > 0


class Socio(Base):
    """Representa un socio de la biblioteca."""
    __tablename__ = 'socios'

    id = Column(Integer, primary_key=True)
    numero_socio = Column(String(20), unique=True, nullable=False)
    documento = Column(String(20), unique=True, nullable=False)
    nombre = Column(String(100), nullable=False)
    apellido = Column(String(100), nullable=False)
    email = Column(String(100))
    telefono = Column(String(20))
    fecha_alta = Column(Date, default=date.today)
    activo = Column(Integer, default=1)  # 1=activo, 0=inactivo

    # Relación uno a muchos con préstamos
    prestamos = relationship('Prestamo', back_populates='socio')

    def __repr__(self):
        return f"<Socio(numero_socio='{self.numero_socio}')>"

    def __str__(self):
        return f"{self.nombre} {self.apellido} (N° {self.numero_socio})"

    @property
    def tiene_prestamos_vencidos(self):
        """Verifica si el socio tiene préstamos vencidos."""
        hoy = date.today()
        return any(p.fecha_devolucion_estimada < hoy and not p.fecha_devolucion_real
                   for p in self.prestamos)

    @property
    def prestamos_activos(self):
        """Retorna la lista de préstamos activos (no devueltos)."""
        return [p for p in self.prestamos if not p.fecha_devolucion_real]


class Prestamo(Base):
    """Representa un préstamo de un libro a un socio."""
    __tablename__ = 'prestamos'

    id = Column(Integer, primary_key=True)
    libro_id = Column(Integer, ForeignKey('libros.id'), nullable=False)
    socio_id = Column(Integer, ForeignKey('socios.id'), nullable=False)
    fecha_prestamo = Column(Date, default=date.today, nullable=False)
    fecha_devolucion_estimada = Column(Date, nullable=False)
```

```
    fecha_devolucion_real = Column(Date)
    observaciones = Column(String(500))

    # Relaciones
    libro = relationship('Libro', back_populates='prestamos')
    socio = relationship('Socio', back_populates='prestamos')

    def __repr__(self):
        return f"<Prestamo(libro_id={self.libro_id}, socio_id={self.socio_id})>"

    def __str__(self):
        estado = "Devuelto" if self.fecha_devolucion_real else "Activo"
        return f"Préstamo #{self.id} - {self.libro.titulo} - {estado}"

    @property
    def esta_vencido(self):
        """Verifica si el préstamo está vencido."""
        if self.fecha_devolucion_real:
            return False
        return date.today() > self.fecha_devolucion_estimada

    @property
    def dias_restantes(self):
        """Calcula los días restantes para la devolución."""
        if self.fecha_devolucion_real:
            return 0
        delta = self.fecha_devolucion_estimada - date.today()
        return delta.days


# Funciones de inicialización

def crear_tablas():
    """Crea todas las tablas en la base de datos."""
    Base.metadata.create_all(engine)


def cargar_datos_ejemplo():
    """Carga datos de ejemplo en la base de datos.
    Devuelve True si cargó los datos o False si la base ya tenía datos."""
    with obtener_session() as session:
        # Verificar si ya hay datos
        if session.query(Autor).count() > 0:
            return False

        # Crear autores
        autor1 = Autor(nombre='Gabriel', apellido='García Márquez',
        ↪  nacionalidad='Colombiano')
        autor2 = Autor(nombre='Jorge', apellido='Luis Borges',
        ↪  nacionalidad='Argentino')
        autor3 = Autor(nombre='Isabel', apellido='Allende', nacionalidad='Chilena')
        autor4 = Autor(nombre='Julio', apellido='Cortázar', nacionalidad='Argentino')

        # Crear libros
        libro1 = Libro(
            isbn='978-950-07-0001-1',
            titulo='Cien años de soledad',
            editorial='Sudamericana',
            anio=1967,
            ejemplares_totales=3,
            ejemplares_disponibles=3
```

```
        )
        libro1.autores.append(autor1)

        libro2 = Libro(
            isbn='978-950-07-0002-8',
            titulo='Ficciones',
            editorial='Emecé',
            anio=1944,
            ejemplares_totales=2,
            ejemplares_disponibles=2
        )
        libro2.autores.append(autor2)

        libro3 = Libro(
            isbn='978-950-07-0003-5',
            titulo='La casa de los espíritus',
            editorial='Plaza & Janés',
            anio=1982,
            ejemplares_totales=2,
            ejemplares_disponibles=2
        )
        libro3.autores.append(autor3)

        libro4 = Libro(
            isbn='978-950-07-0004-2',
            titulo='Rayuela',
            editorial='Sudamericana',
            anio=1963,
            ejemplares_totales=2,
            ejemplares_disponibles=2
        )
        libro4.autores.append(autor4)

        # Crear socios
        socio1 = Socio(
            numero_socio='S001',
            documento='12345678',
            nombre='Ana',
            apellido='Martínez',
            email='ana@example.com',
            telefono='1234-5678'
        )

        socio2 = Socio(
            numero_socio='S002',
            documento='87654321',
            nombre='Carlos',
            apellido='López',
            email='carlos@example.com',
            telefono='8765-4321'
        )

        session.add_all([autor1, autor2, autor3, autor4,
                         libro1, libro2, libro3, libro4,
                         socio1, socio2])

        return True


# DAOs: una clase repositorio por entidad principal.
# No interactúan con el usuario: solo acceden a los datos y devuelven
```

```python
# valores (objetos, diccionarios o None). La interacción (input/print)
# queda a cargo del main.
#
# Los métodos que devuelven datos para mostrar los materializan dentro de
# la sesión (como diccionarios), porque las relaciones lazy no pueden
# consultarse una vez cerrada la sesión (DetachedInstanceError).


class LibroDAO:
    """Operaciones de acceso a datos sobre la tabla de libros."""

    def obtener_por_isbn(self, isbn):
        """Devuelve el libro con el ISBN indicado, o None si no existe."""
        with obtener_session() as session:
            return session.query(Libro).filter_by(isbn=isbn).first()

    def listar(self):
        """Devuelve todos los libros con los datos listos para mostrar."""
        with obtener_session() as session:
            libros = session.query(Libro).all()

            return [
                {
                    'isbn': libro.isbn,
                    'titulo': libro.titulo,
                    'autores': ', '.join([str(autor) for autor in libro.autores]),
                    'editorial': libro.editorial,
                    'anio': libro.anio,
                    'disponibles': libro.ejemplares_disponibles,
                    'totales': libro.ejemplares_totales,
                }
                for libro in libros
            ]


class SocioDAO:
    """Operaciones de acceso a datos sobre la tabla de socios."""

    def obtener_por_numero(self, numero_socio):
        """Devuelve el socio con el número indicado, o None si no existe."""
        with obtener_session() as session:
            return session.query(Socio).filter_by(numero_socio=numero_socio).first()

    def listar_activos(self):
        """Devuelve los socios activos con los datos listos para mostrar."""
        with obtener_session() as session:
            socios = session.query(Socio).filter_by(activo=1).all()

            return [
                {
                    'numero_socio': socio.numero_socio,
                    'nombre': socio.nombre,
                    'apellido': socio.apellido,
                    'documento': socio.documento,
                    'email': socio.email,
                    'prestamos_activos': len(socio.prestamos_activos),
                }
                for socio in socios
            ]
```

```
class PrestamoDAO:
    """Operaciones de acceso a datos sobre la tabla de préstamos."""

    def registrar(self, numero_socio, isbn, dias_prestamo=14):
        """Registra un nuevo préstamo.
        Lanza ValueError si no se cumplen las reglas de negocio."""
        with obtener_session() as session:
            # Buscar socio
            socio = session.query(Socio).filter_by(numero_socio=numero_socio).first()
            if not socio:
                raise ValueError(f"Socio {numero_socio} no encontrado")

            if not socio.activo:
                raise ValueError(f"Socio {numero_socio} no está activo")

            if socio.tiene_prestamos_vencidos:
                raise ValueError(f"Socio {numero_socio} tiene préstamos vencidos")

            # Buscar libro
            libro = session.query(Libro).filter_by(isbn=isbn).first()
            if not libro:
                raise ValueError(f"Libro con ISBN {isbn} no encontrado")

            if not libro.disponible:
                raise ValueError(f"No hay ejemplares disponibles de '{libro.titulo}'")

            # Crear préstamo
            fecha_devolucion = date.today() + timedelta(days=dias_prestamo)
            prestamo = Prestamo(
                libro=libro,
                socio=socio,
                fecha_devolucion_estimada=fecha_devolucion
            )

            # Actualizar disponibilidad
            libro.ejemplares_disponibles -= 1

            session.add(prestamo)

            return {
                'titulo': libro.titulo,
                'socio': f"{socio.nombre} {socio.apellido}",
                'fecha_devolucion': fecha_devolucion,
            }

    def devolver(self, prestamo_id, observaciones=None):
        """Registra la devolución de un préstamo.
        Lanza ValueError si el préstamo no existe o ya fue devuelto."""
        with obtener_session() as session:
            prestamo = session.query(Prestamo).get(prestamo_id)
            if not prestamo:
                raise ValueError(f"Préstamo #{prestamo_id} no encontrado")

            if prestamo.fecha_devolucion_real:
                raise ValueError(f"El préstamo #{prestamo_id} ya fue devuelto")

            # Registrar devolución
            prestamo.fecha_devolucion_real = date.today()
            if observaciones:
                prestamo.observaciones = observaciones
```

```
            # Actualizar disponibilidad
            prestamo.libro.ejemplares_disponibles += 1

            return {
                'titulo': prestamo.libro.titulo,
                'vencido': prestamo.esta_vencido,
            }

    def consultar_por_socio(self, numero_socio):
        """Devuelve el socio y sus préstamos listos para mostrar.
        Lanza ValueError si el socio no existe."""
        with obtener_session() as session:
            socio = session.query(Socio).filter_by(numero_socio=numero_socio).first()
            if not socio:
                raise ValueError(f"Socio {numero_socio} no encontrado")

            prestamos = []
            for prestamo in socio.prestamos:
                estado = "DEVUELTO" if prestamo.fecha_devolucion_real else "ACTIVO"
                if prestamo.esta_vencido:
                    estado = "VENCIDO"

                prestamos.append({
                    'id': prestamo.id,
                    'titulo': prestamo.libro.titulo,
                    'fecha_prestamo': prestamo.fecha_prestamo,
                    'fecha_devolucion_estimada': prestamo.fecha_devolucion_estimada,
                    'fecha_devolucion_real': prestamo.fecha_devolucion_real,
                    'estado': estado,
                    'dias_restantes': prestamo.dias_restantes,
                    'observaciones': prestamo.observaciones,
                })

            return {
                'nombre': socio.nombre,
                'apellido': socio.apellido,
                'prestamos': prestamos,
            }

    def listar_activos(self):
        """Devuelve los préstamos activos listos para mostrar."""
        with obtener_session() as session:
            prestamos = session.query(Prestamo).filter(
                Prestamo.fecha_devolucion_real.is_(None)
            ).all()

            return [
                {
                    'id': prestamo.id,
                    'titulo': prestamo.libro.titulo,
                    'socio': f"{prestamo.socio.nombre} {prestamo.socio.apellido}",
                    'fecha_devolucion_estimada': prestamo.fecha_devolucion_estimada,
                    'estado': "VENCIDO" if prestamo.esta_vencido else "VIGENTE",
                    'dias_restantes': prestamo.dias_restantes,
                }
                for prestamo in prestamos
            ]


# Menú principal: aquí vive toda la interacción con el usuario
```

```
def mostrar_menu():
    """Muestra el menú principal."""
    print("\n" + "="*50)
    print("SISTEMA DE GESTIÓN DE BIBLIOTECA")
    print("="*50)
    print("1. Listar libros")
    print("2. Listar socios")
    print("3. Registrar préstamo")
    print("4. Registrar devolución")
    print("5. Consultar préstamos de un socio")
    print("6. Listar préstamos activos")
    print("7. Cargar datos de ejemplo")
    print("0. Salir")
    print("="*50)


def mostrar_catalogo(libros):
    """Muestra el catálogo de libros."""
    print("\n═══ CATÁLOGO DE LIBROS ═══")
    for libro in libros:
        print(f"\nISBN: {libro['isbn']}")
        print(f"Título: {libro['titulo']}")
        print(f"Autor(es): {libro['autores']}")
        print(f"Editorial: {libro['editorial']} ({libro['anio']})")
        print(f"Disponibles: {libro['disponibles']}/{libro['totales']}")


def mostrar_socios(socios):
    """Muestra los socios activos."""
    print("\n═══ SOCIOS ACTIVOS ═══")
    for socio in socios:
        print(f"\nN° Socio: {socio['numero_socio']}")
        print(f"Nombre: {socio['nombre']} {socio['apellido']}")
        print(f"Documento: {socio['documento']}")
        print(f"Email: {socio['email']}")
        print(f"Préstamos activos: {socio['prestamos_activos']}")


def mostrar_prestamos_socio(datos):
    """Muestra los préstamos de un socio."""
    print(f"\n═══ PRÉSTAMOS DE {datos['nombre']} {datos['apellido']} ═══")

    if not datos['prestamos']:
        print("No tiene préstamos registrados")
        return

    for prestamo in datos['prestamos']:
        print(f"\nPréstamo #{prestamo['id']} - {prestamo['estado']}")
        print(f"Libro: {prestamo['titulo']}")
        print(f"Fecha préstamo: {prestamo['fecha_prestamo']}")
        print(f"Fecha devolución estimada: {prestamo['fecha_devolucion_estimada']}")

        if prestamo['fecha_devolucion_real']:
            print(f"Fecha devolución real: {prestamo['fecha_devolucion_real']}")
        else:
            print(f"Días restantes: {prestamo['dias_restantes']}")

        if prestamo['observaciones']:
            print(f"Observaciones: {prestamo['observaciones']}")
```

```
def mostrar_prestamos_activos(prestamos):
    """Muestra los préstamos activos."""
    print("\n═══ PRÉSTAMOS ACTIVOS ═══")

    if not prestamos:
        print("No hay préstamos activos")
        return

    for prestamo in prestamos:
        print(f"\nPréstamo #{prestamo['id']} - {prestamo['estado']}")
        print(f"Libro: {prestamo['titulo']}")
        print(f"Socio: {prestamo['socio']}")
        print(f"Fecha devolución: {prestamo['fecha_devolucion_estimada']}")
        print(f"Días restantes: {prestamo['dias_restantes']}")


def main():
    """Función principal."""
    crear_tablas()
    print("Tablas creadas exitosamente")

    libro_dao = LibroDAO()
    socio_dao = SocioDAO()
    prestamo_dao = PrestamoDAO()

    while True:
        mostrar_menu()
        opcion = input("\nSeleccione una opción: ").strip()

        try:
            if opcion == '1':
                mostrar_catalogo(libro_dao.listar())

            elif opcion == '2':
                mostrar_socios(socio_dao.listar_activos())

            elif opcion == '3':
                numero_socio = input("Número de socio: ").strip()
                isbn = input("ISBN del libro: ").strip()
                datos = prestamo_dao.registrar(numero_socio, isbn)
                print(f"Préstamo registrado: {datos['titulo']} para {datos['socio']}")
                print(f"Fecha de devolución: {datos['fecha_devolucion']}")

            elif opcion == '4':
                prestamo_id = input("ID del préstamo: ").strip()
                observaciones = input("Observaciones (opcional): ").strip()
                try:
                    prestamo_id = int(prestamo_id)
                except ValueError:
                    print("Error: ID inválido")
                else:
                    datos = prestamo_dao.devolver(prestamo_id, observaciones or None)
                    if datos['vencido']:
                        print("Advertencia: Devolución con retraso")
                    print(f"Devolución registrada: {datos['titulo']}")

            elif opcion == '5':
                numero_socio = input("Número de socio: ").strip()
                mostrar_prestamos_socio(
                    prestamo_dao.consultar_por_socio(numero_socio)
```

```
                )

            elif opcion == '6':
                mostrar_prestamos_activos(prestamo_dao.listar_activos())

            elif opcion == '7':
                if cargar_datos_ejemplo():
                    print("Datos de ejemplo cargados exitosamente")
                else:
                    print("La base de datos ya contiene datos")

            elif opcion == '0':
                print("¡Hasta luego!")
                break

            else:
                print("Opción inválida")
        except ValueError as e:
            print(f"Error: {e}")

        input("\nPresione Enter para continuar...")


if __name__ == '__main__':
    main()
```